

GestorAdv — Documentação técnica completa

Versão consolidada (monorepo: API NestJS, web Next.js, Prisma, modos standalone e SaaS).

Documento gerado automaticamente em 16 de abril de 2026.

Índice e referência rápida

Índice da documentação do monorepo **GestorAdv** (gestão para escritórios de advocacia).

Documento	Conteúdo
Visão geral	Objetivo do produto, escopo e princípios
Arquitetura	Monorepo, modos de deploy, fluxos
Stack tecnológica	Versões e dependências principais
Banco de dados	Prisma, schema tenant vs master, comandos
API (backend)	NestJS, módulos, Swagger, saúde
Frontend web	Next.js App Router, rotas, variáveis
Autenticação e multitenancy	Auth, licenças, SaaS vs standalone
Variáveis de ambiente	Referência de <code>.env</code>
Instalação e execução	Dev, Docker, build
Integrações	Filas, WhatsApp, tribunais, e-mail
Ferramentas e serviços auxiliares	License manager, provisioner, <code>services/</code>

Referência rápida

- API REST (prefixo global): `/api`
- Documentação interativa: `http://<host>:<PORT>/api/docs` (Swagger)
- Front padrão dev: porta **3000**; API padrão dev: porta **3001**

Documentos de apresentação comercial existentes em `portfolio-gestoradv-*` permanecem separados desta documentação técnica.

Visão geral

Objetivo

O **GestorAdv** é um sistema para **gestão de escritórios de advocacia**, com foco em processos, clientes, prazos, financeiro, atendimento, relatórios e integrações (comunicação e tribunais). O código está organizado como **monorepo** TypeScript, com API em NestJS e interface web em Next.js.

Escopo técnico atual

- **Backend:** API REST (`apps/api`), validação com `class-validator` , documentação OpenAPI (Swagger).
- **Frontend:** SPA/SSR com Next.js App Router (`apps/web`).
- **Dados:** PostgreSQL (Prisma), Redis (filas BullMQ), MongoDB (variáveis de ambiente previstas; uso conforme módulos).
- **Modos de operação:** instalação **standalone** (um escritório por base) ou **SaaS multi-tenant** (banco master + tenants com bancos dedicados), configurável por ambiente.

Conformidade e segurança

O modelo de dados inclui campos de LGPD (consentimento, rastreabilidade em auditoria onde aplicável). A API usa **Helmet**, **CORS** restrito à origem configurada em `NEXTAUTH_URL` (nome herdado do ecossistema; o login web atual usa **JWT** retornado por `POST /api/auth/login` e estado no cliente). **ValidationPipe** global e guards JWT nas rotas protegidas. Detalhes em Autenticação e multitenancy.

Documentação de produto vs técnica

- Esta pasta (`docs/*.md`) descreve **implementação e operação**.
- Arquivos `portfolio-gestoradv-*` e `contexto.txt` são materiais de contexto/apresentação e podem antecipar funcionalidades ainda não refletidas integralmente no código.

Arquitetura

Monorepo

O repositório usa **pnpm workspaces** e **Turborepo** para orquestrar builds e `dev`.

```
GestorAdv/
├─ apps/
│  ├─ api/          # NestJS – API REST
│  └─ web/          # Next.js – interface web
├─ packages/
│  ├─ database/     # Prisma (schema tenant + master)
│  ├─ tsconfig/     # Bases TypeScript compartilhadas
│  └─ validators/   # Schemas Zod compartilhados
├─ services/        # Pacotes auxiliares (chatbot, scraper, document-generator)
├─ tools/           # Scripts operacionais (license-manager, tenant-provisioner)
├─ docker-compose.yml
├─ package.json
├─ pnpm-workspace.yaml
└─ turbo.json
```

Workspaces declarados em `pnpm-workspace.yaml`: `apps/*`, `packages/*`, `services/*`.

Componentes em runtime

Fluxo de componentes (resumo para impressão):

```
graph TD
    Navegador --> NextjsWeb[Next.js web]
    NextjsWeb --> NestJSAPI[NestJS API]
    NestJSAPI --> PGtenant[PostgreSQL tenant]
    NestJSAPI --> Redis
    PGtenant --> PGmaster[PostgreSQL master, SaaS]
    Redis --> MongoDB[MongoDB]
```

Standalone: um banco tenant com licença no modelo Escritório.

SaaS: banco master (Tenant, TenantLicense) + banco(s) por tenant.

(Diagrama Mermaid interativo: `docs/02-arquitetura.md` no repositório.)

- **Standalone**: normalmente um único PostgreSQL “tenant” (`DATABASE_URL`) com tabela `Escritorio` e licença local; `MASTER_DATABASE_URL` pode existir para ferramentas, mas o fluxo principal de licença é o do escritório.
- **SaaS**: PostgreSQL **master** com `Tenant`, `TenantLicense`, `TenantPlan`; cada tenant pode ter `databaseUrl` próprio; o middleware resolve o tenant e o `LicenseGuard` valida licença no master.

Modos de deploy

Modo	Variável	Comportamento resumido
Standalone	<code>DEPLOYMENT_MODE=standalone</code> (padrão em <code>.env.example</code>)	Licença no registro <code>Escritorio</code> do banco tenant; sem exigência de subdomínio/header de tenant nas rotas normais.
SaaS	<code>DEPLOYMENT_MODE=saas</code>	Middleware de tenant ativo; identificação por <code>x-Tenant-ID</code> (slug) ou subdomínio; licença em <code>TenantLicense</code> no master.

Detalhes em Autenticação e multitenancy.

API global

- Prefixo: `/api` (ex.: `/api/auth/login`).
- Guard global: `LicenseGuard` (com exceções para health, auth, escritório, webhooks, admin, etc.).
- Swagger UI: `/api/docs` .

Pool de conexões multi-tenant

O serviço `TenantPrismaService` mantém um pool de clientes Prisma por `databaseUrl` , com limpeza de conexões ociosas. Em modo SaaS, isso sustenta um banco dedicado por escritório quando configurado no cadastro do tenant.

Stack tecnológica

Valores abaixo referem-se ao estado atual do repositório (consulte `package.json` nos pacotes para versões exatas).

Runtime e tooling

Tecnologia	Uso
Node.js	<code>>= 20</code> (raiz <code>package.json</code> <code>engines</code>)
pnpm	<code>10.33.0</code> (<code>packageManager</code>)
TypeScript	Tipagem estrita nos apps e packages
Turborepo	Build e <code>turbo dev</code> na raiz
Prettier	Formatação

Backend (`apps/api`)

Pacote	Função
NestJS	Framework HTTP, módulos, injeção de dependências
@nestjs/swagger	OpenAPI / Swagger
Passport (JWT, local)	Autenticação
class-validator / class-transformer	DTOs e validação
Helmet	Cabeçalhos de segurança HTTP
BullMQ	Filas assíncronas (Redis)
Nodemailer	E-mail

Frontend (`apps/web`)

Pacote	Função
Next.js (~15)	App Router, SSR/SSG
React (~19)	UI
TanStack Query	Cache e requisições

Pacote	Função
React Hook Form + Zod	Formulários
Tailwind CSS (~4)	Estilos

Dados (packages/database)

Pacote	Função
Prisma	ORM, dois schemas: tenant e master

Infraestrutura local

Serviço	Imagem / uso
PostgreSQL	16-alpine (docker-compose.yml)
Redis	7-alpine
MongoDB	7

Serviços auxiliares (services/)

Pacotes Node separados (ex.: chatbot com triagem, scraper Playwright, gerador de peças). Dependências próprias em cada `package.json` ; não são necessariamente iniciados pelo `turbo dev` da raiz — ver Ferramentas e serviços auxiliares.

Banco de dados

Dois schemas Prisma

O pacote `packages/database` define:

1. **Schema tenant** — `prisma/schema.prisma`
 - Variável: `DATABASE_URL`
 - Dados do escritório: usuários, clientes, processos, prazos, tarefas, documentos, notificações, financeiro (`Lancamento` , `ContaPagar`), atendimento, auditoria, `Escritorio` (incluindo campos de licença e integrações).
2. **Schema master** — `prisma/master.prisma`
 - Variável: `MASTER_DATABASE_URL`
 - Modelos: `Tenant` , `TenantLicense` , `TenantPlan` , `AdminUser` (painel fornecedor SaaS).

Modelos principais (tenant)

Área	Modelos
Auth	<code>User</code> , <code>Session</code>
CRM	<code>Client</code>
Processual	<code>Processo</code> , <code>Movimentacao</code> , <code>Prazo</code> , <code>Documento</code> , <code>Tarefa</code>
Sistema	<code>Notification</code> , <code>Escritorio</code> , <code>AuditLog</code>
Financeiro	<code>Lancamento</code> , <code>ContaPagar</code>
Atendimento	<code>Atendimento</code> , <code>AtendimentoMensagem</code>

Enums relevantes: `Role` , `ProcessoStatus` , `AreaDireito` , `PrazoStatus` , `LancamentoTipo` , etc.

Modelos principais (master)

Modelo	Descrição
<code>Tenant</code>	Escritório no SaaS: <code>slug</code> , <code>cnpj</code> , <code>databaseUrl</code> , <code>databaseName</code> , flags de ativo/excluído
<code>TenantLicense</code>	Licença por <code>tenantId</code> : chave, plano, validade, ativa
<code>TenantPlan</code>	Planos com limites (usuários, processos, armazenamento, flags de WhatsApp/IA)

Modelo	Descrição
AdminUser	Usuários do painel administrativo do fornecedor

Comandos Prisma (pacote database)

Executar a partir de `packages/database` ou via scripts da raiz (`pnpm db:*`):

Script	Descrição
<code>generate</code>	Cliente Prisma schema tenant
<code>generate:master</code>	Cliente Prisma schema master
<code>generate:all</code>	Ambos
<code>push</code> / <code>push:master</code> / <code>push:all</code>	<code>db push</code> (dev; sincroniza schema sem migration formal)
<code>migrate</code>	<code>prisma migrate dev</code> (tenant)
<code>studio</code> / <code>studio:master</code>	Prisma Studio
<code>seed</code> / <code>seed:master</code>	Seeds (quando implementados)
<code>migrate:tenant</code>	Script <code>tools/tenant-provisioner/migrate-existing.ts</code>

A raiz expõe atalhos: `pnpm db:generate` , `pnpm db:push` , `pnpm db:migrate` , `pnpm db:studio` (ver `package.json`).

Bancos lógicos no Docker

O `docker-compose.yml` cria um único cluster PostgreSQL com banco padrão `gestoradv` . Em desenvolvimento costuma-se criar também o banco `gestoradv_master` (conforme `MASTER_DATABASE_URL` no `.env.example`) manualmente ou via script, para o schema master.

Nota: o `docker-compose` atual pode usar volumes com caminho absoluto da máquina de desenvolvimento; em outro ambiente, ajuste os volumes ou use volumes nomeados (ver Instalação e execução).

API (backend)

Entrada

- Arquivo principal: `apps/api/src/main.ts`
- Prefixo global: `api` → rotas como `/api/auth/login`
- Porta padrão: `PORT` ou `3001`
- CORS: origem `NEXTAUTH_URL` ou `http://localhost:3000`
- Swagger: `GET /api/docs`

Módulos NestJS

Importados em `app.module.ts` :

Módulo	Responsabilidade
<code>ConfigModule</code>	Variáveis de ambiente globais
<code>MasterPrismaModule</code>	Cliente Prisma ao banco master
<code>PrismaModule</code>	Cliente Prisma tenant (singleton)
<code>TenantModule</code>	Middleware SaaS + contexto + pool multi-DB
<code>AuthModule</code>	Login/registro
<code>UsersModule</code>	Usuários
<code>ProcessosModule</code>	Processos
<code>ClientsModule</code>	Clientes
<code>PrazosModule</code>	Prazos
<code>TarefasModule</code>	Tarefas
<code>NotificationsModule</code>	Notificações
<code>QueuesModule</code>	BullMQ / Redis
<code>EmailModule</code>	Envio de e-mail
<code>UploadModule</code>	Upload de documentos
<code>ChatbotModule</code>	Chatbot
<code>AtendimentoModule</code>	Atendimentos + rotas públicas de formulário
<code>EscritorioModule</code>	Dados do escritório e licença (standalone)

Módulo	Responsabilidade
FinanceiroModule	Financeiro e contas a pagar
RelatoriosModule	Relatórios
WhatsappModule	Webhooks WhatsApp
CalendarModule	Calendário
TribunaisModule	Integração tribunais/DATAJUD
AdminModule	Painel fornecedor (tenants, planos, licenças)

Controllers e prefixos REST

Prefixo base da API: `/api`. Caminhos relativos ao prefixo Nest `api`:

Prefixo controller	Exemplo
health	Saúde da API
auth	POST <code>/auth/login</code> , POST <code>/auth/register</code>
users	CRUD usuários
processos	Processos
clients	Clientes
prazos	Prazos
tarefas	Tarefas
notifications	Notificações
documents	Upload/documentos
chatbot	Chatbot
atendimentos	Atendimentos autenticados
atendimento	Rotas públicas (formulário)
escritorio	Configuração do escritório
financeiro, contas-pagar	Financeiro
relatorios	Relatórios
webhooks/whatsapp	Webhook Meta

Prefixo controller	Exemplo
<code>tribunais</code>	Tribunais
<code>admin</code>	Autenticação admin, tenants, planos, licenças, dashboard

Guard de licença

`LicenseGuard` está registrado como `APP_GUARD` global. Rotas abertas (sem validação de licença de escritório) incluem, entre outras, prefixos como `/api/health`, `/api/auth`, `/api/escritorio`, `/api/webhooks`, `/api/atendimento/formulario`, `/api/admin` — ver implementação em `license.guard.ts`.

SaaS: identificação de tenant

Com `DEPLOYMENT_MODE=saas`, o `TenantMiddleware` exige:

- header `X-Tenant-ID` com valor igual ao **slug** do tenant, ou
- hostname com **subdomínio** (3+ partes; primeiro segmento = slug).

Rotas `/api/admin` e `/api/health` não passam pela resolução obrigatória de tenant da mesma forma (ver código).

Documentação OpenAPI

Gerada a partir dos decorators `@ApiTags`, `@ApiOperation`, etc. Acesse `/api/docs` em ambiente de desenvolvimento para lista completa e testes.

Frontend web

Stack

- **Next.js** App Router — diretório `apps/web/src/app`
- Estilos: **Tailwind CSS**
- Estado: **Zustand** (ex.: sessão do usuário e token JWT após login)
- Dados: **TanStack Query** + `fetch` para a API
- Formulários: **React Hook Form** + **Zod** (validadores em `@gestor-adv/validators` quando aplicável)

Porta e scripts

- Desenvolvimento: `next dev --port 3000` (`apps/web/package.json`)
- Produção: `next build` / `next start` (porta configurável pelo host)

URL da API

O front usa a variável `NEXT_PUBLIC_API_URL` , com fallback `http://localhost:3001/api` (ex.: `apps/web/src/lib/api.ts`).

Importante: em produção ou ao usar hostname customizado, defina `NEXTAUTH_URL` e `NEXT_PUBLIC_API_URL` de forma consistente com a URL que o usuário acessa no navegador.

Mapa de rotas (App Router)

Rota	Função típica
<code>/</code>	Home
<code>/login</code>	Login do escritório
<code>/register</code>	Cadastro
<code>/landing</code>	Landing
<code>/dashboard</code>	Painel principal
<code>/dashboard/processos</code>	Processos
<code>/dashboard/processos/[id]</code>	Detalhe do processo
<code>/dashboard/clientes</code>	Clientes
<code>/dashboard/prazos</code>	Prazos

Rota	Função típica
/dashboard/tarefas	Tarefas
/dashboard/documentos	Documentos
/dashboard/financeiro	Financeiro
/dashboard/escritorio	Dados e integrações do escritório
/dashboard/advogados	Advogados
/dashboard/agenda	Agenda
/dashboard/relatorios	Relatórios
/dashboard/chatbot	Chatbot
/atendimento	Atendimento (área específica)
/admin/login	Login do painel fornecedor
/admin	Dashboard admin
/admin/escritorios	Lista de tenants
/admin/escritorios/[id]	Detalhe/ licença do tenant
/admin/licencas	Visão de licenças
/admin/planos	Planos SaaS

Painel administrativo

Rotas sob `/admin/*` consomem a API `/api/admin/*` com autenticação de administrador (JWT admin). Ver `apps/web/src/lib/admin-api.ts`.

Modo SaaS no browser

Para requisições à API em modo SaaS, o backend espera **subdomínio** ou header `X-Tenant-ID`. O front pode precisar de configuração adicional (proxy, variáveis por ambiente ou middleware Next) para enviar o tenant em todas as chamadas — validar cenário de deploy antes de ir a produção.

Autenticação e multitenancy

Autenticação de usuários do escritório

- **Endpoints:** `AuthController` — `POST /api/auth/login` (Passport local), `POST /api/auth/register`
- **JWT:** o login devolve `access_token`; o front (`login/page.tsx`) armazena o token (ex.: Zustand) e envia **Bearer** nas requisições à API
- **Sessões:** modelo `Session` existe no Prisma tenant para extensões futuras ou fluxos híbridos; o fluxo principal da web atual é JWT em memória/localStorage conforme `auth store`

`NEXTAUTH_URL` : usado pela API em **CORS** (`origin`) e convém coincidir com a URL base do front.

`NEXTAUTH_SECRET` está no `.env.example` para alinhar com stacks Auth.js/NextAuth se forem adotados no futuro; o login atual não depende do pacote `next-auth` no `apps/web` .

Autenticação admin (fornecedor)

- **Endpoint:** `POST /api/admin/auth/login`
- Rotas subsequentes em `/api/admin/*` usam `AdminAuthGuard` e Bearer token de admin.

Licenciamento — modo standalone

- Dados no modelo `Escritorio` : `licencaChave` , `licencaValidade` , `licencaPlano` , `licencaAtiva` , histórico.
- `LicenseGuard` : chama `EscritorioService.checkAndRevalidate()` com cache de alguns minutos; bloqueia rotas protegidas se inválido.
- Ferramenta CLI: `tools/license-manager` — geração/validação/renovação por CNPJ.

Licenciamento — modo SaaS

- Com `DEPLOYMENT_MODE=saas` , o guard consulta `TenantLicense` no banco **master** pelo `tenantId` injetado no request.
- Requisitos: licença existente, `ativa=true` , `validade` futura.
- Gestão via API admin: gerar, revogar, renovar licença por tenant.

Resolução de tenant (SaaS)

`TenantMiddleware` :

1. Ignora certos prefixos (ex.: `/api/admin` , `/api/health`).
2. Obtém **slug** via `X-Tenant-ID` ou subdomínio.
3. Carrega `Tenant` no master; valida ativo/não excluído.
4. Preenche `TenantContext` e anexa `tenantId` ao request.

`TenantPrismaService` fornece instâncias `PrismaClient` por `databaseUrl` para isolamento por banco.

Rotas sem checagem de licença de escritório

Lista atual (trecho lógico) em `LicenseGuard` : health, auth, escritório (configuração), webhooks, formulário público de atendimento, admin. Ajustar com cuidado para não expor operações sensíveis.

Variáveis de ambiente

Referência baseada em `.env.example`. Copie para `.env` na raiz e/ou em `apps/api` conforme o fluxo de carregamento do projeto.

Banco e multitenancy

Variável	Descrição
<code>DATABASE_URL</code>	PostgreSQL schema tenant
<code>MASTER_DATABASE_URL</code>	PostgreSQL schema master (SaaS)
<code>MONGODB_URI</code>	MongoDB
<code>REDIS_URL</code>	Redis (filas BullMQ)
<code>DEPLOYMENT_MODE</code>	<code>standalone</code> (padrão) ou <code>saas</code>
<code>TENANT_DB_HOST</code> / <code>TENANT_DB_PORT</code> / <code>TENANT_DB_USER</code> / <code>TENANT_DB_PASSWORD</code>	Usados em provisionamento/admin ao criar bancos de tenant

IA

Variável	Descrição
<code>ANTHROPIC_API_KEY</code>	Claude
<code>OPENAI_API_KEY</code>	OpenAI
<code>PINECONE_API_KEY</code>	Vetores (se RAG)

WhatsApp (fallback global)

Credenciais também podem ser armazenadas por escritório no banco (`Escritorio`).

Variável	Descrição
<code>WHATSAPP_BUSINESS_ID</code>	ID do negócio
<code>WHATSAPP_ACCESS_TOKEN</code>	Token de acesso
<code>WHATSAPP_PHONE_NUMBER_ID</code>	ID do número
<code>WHATSAPP_VERIFY_TOKEN</code>	Verificação do webhook

E-mail

Variável	Descrição
<code>SENDGRID_API_KEY</code>	SendGrid
<code>SMTP_HOST</code> / <code>SMTP_PORT</code>	SMTP genérico

Google / Meta

Variável	Descrição
<code>GOOGLE_CLIENT_ID</code> / <code>GOOGLE_CLIENT_SECRET</code>	OAuth Google
<code>GOOGLE_ADS_DEVELOPER_TOKEN</code>	Google Ads
<code>GOOGLE_CALENDAR_API_KEY</code>	Calendário
<code>META_APP_ID</code> / <code>META_APP_SECRET</code> / <code>META_ACCESS_TOKEN</code>	Meta

Autenticação web/API

Variável	Descrição
<code>NEXTAUTH_SECRET</code>	Reservado / futuro Auth.js; gerar valor forte
<code>NEXTAUTH_URL</code>	URL pública do front — origem CORS da API (<code>main.ts</code>)
<code>JWT_SECRET</code>	Assinatura JWT dos usuários da API
<code>ADMIN_JWT_SECRET</code>	Assinatura JWT do painel admin

Monitoramento e scraping

Variável	Descrição
<code>SENTRY_DSN</code>	Sentry
<code>LOGTAIL_TOKEN</code>	Logs
<code>PROXY_URL</code>	Proxy scraping
<code>TWOCAPTCHA_API_KEY</code>	Captcha

API (porta)

Variável	Descrição
PORT	Porta da API Nest (padrão 3001)

Front (build)

Prefixo `NEXT_PUBLIC_*` é embutido no bundle do Next.js em build time.

Variável	Descrição
NEXT_PUBLIC_API_URL	Base da API vista pelo browser (ex.: <code>https://api.exemplo.com/api</code>)

Instalação e execução

Pré-requisitos

- **Node.js** ≥ 20
- **pnpm** (versão fixada no `package.json` raiz)
- **Docker** + Docker Compose (recomendado para Postgres, Redis e MongoDB locais)

Clonar e instalar dependências

```
pnpm install
```

Na raiz do monorepo.

Variáveis de ambiente

1. Copie `.env.example` para `.env` na raiz (e ajuste `apps/api` se o Nest carregar `.env` local).
2. Gere segredos fortes para `NEXTAUTH_SECRET`, `JWT_SECRET`, `ADMIN_JWT_SECRET`.
3. Ajuste `NEXTAUTH_URL` e `NEXT_PUBLIC_API_URL` para a URL real do front e da API.

Banco de dados com Docker

```
pnpm docker:up
```

Equivale a `docker compose up -d` (`package.json`).

Serviços definidos em `docker-compose.yml`:

- **PostgreSQL** — porta `5432`, usuário/senha/db conforme compose
- **Redis** — `6379`
- **MongoDB** — `27017`

Volumes: o arquivo atual usa caminho absoluto Windows (`d:/sistemas/GestorAdv/.docker-data/...`). Em outra máquina ou SO, altere para um caminho válido ou use **volumes nomeados** do Docker.

Crie o banco `gestoradv_master` no Postgres se usar o schema master (ex.: `CREATE DATABASE gestoradv_master;`).

Prisma

Gerar clientes e aplicar schema (desenvolvimento):

```
pnpm db:generate
pnpm db:push
```

Para master também:

```
pnpm --filter @gestor-adv/database generate:master
pnpm --filter @gestor-adv/database push:master
```

(ou scripts `generate:all` / `push:all` dentro do pacote `database`).

Desenvolvimento

Na raiz:

```
pnpm dev
```

Turborepo sobe os apps configurados (API + web). Portas típicas: **3000** (web), **3001** (API).

Build de produção

```
pnpm build
```

Executar `turbo build` nos workspaces. Para servir: `next start` no web e `node dist/main` na API (após `nest build`), conforme scripts de cada app.

Acesso

- Front: `http://localhost:3000` (ou URL configurada)
- API: `http://localhost:3001/api`
- Swagger: `http://localhost:3001/api/docs`

Nome de host em vez de localhost

Para usar um hostname (ex.: `gestoradv.local`), configure DNS interno ou o arquivo `hosts` e alinhe `NEXTAUTH_URL` e `NEXT_PUBLIC_API_URL` — ver Frontend web.

Integrações

Filas (Redis / BullMQ)

Módulo `QueuesModule` :

- Conexão Redis derivada de `REDIS_URL` (host/porta).
- Fila registrada: `prazos-alerts` — processador `PrazosAlertProcessor` para alertas de prazos (`PrazosAlertService`).

WhatsApp

- **Webhook:** controller em `webhooks/whatsapp` (`WhatsappController`).
- Credenciais podem vir do `.env` global ou dos campos do modelo `Escritorio` no banco (integrações por escritório), conforme implementação do `WhatsappService`.

Tribunais / DATAJUD

- Módulo `TribunaisModule` — API para consultas; chave pode ser configurada no escritório (`datajudApiKey` em `Escritorio`).

E-mail

- `EmailModule` — envio via Nodemailer com configuração SMTP/SendGrid a partir do ambiente.

Calendário

- `CalendarModule` — integração Google Calendar (depende de chaves no `.env`).

Upload de documentos

- `UploadModule` — rotas sob prefixo `documents` para upload/armazenamento de arquivos do escritório.

Chatbot

- `ChatbotModule` — endpoints REST do assistente; pode depender de serviços de IA configurados no ambiente.

Serviços separados (`services/`)

Pacotes opcionais com responsabilidades específicas (triagem, scraping com Playwright, geração de peças). Não fazem parte do processo único `pnpm dev` da raiz salvo configuração adicional — ver Ferramentas e serviços auxiliares.

Webhooks e segurança

Exponha webhooks apenas com HTTPS em produção e valide tokens (ex.: `WHATSAPP_VERIFY_TOKEN`). Mantenha segredos fora do repositório.

Ferramentas e serviços auxiliares

tools/license-manager

Script Node para **gerar, validar, renovar e revogar licenças** no modo standalone, operando diretamente no PostgreSQL do tenant.

- Documentação: `tools/license-manager/README.md`
- Requer `DATABASE_URL` apontando ao banco do escritório.
- Fluxo: trial automático na criação do escritório; após expiração, geração de chave por CNPJ e ativação na interface.

tools/tenant-provisioner

Script `migrate-existing.ts` para cenários de migração/provisionamento com `MASTER_DATABASE_URL` e instruções para `DEPLOYMENT_MODE=saas`.

Comando relacionado no pacote database: `pnpm migrate:tenant` (no pacote `@gestor-adv/database`).

services/chatbot

Serviço de triagem/conversa (`src/triagem.service.ts` , `prompts.ts`). Execução via scripts do próprio pacote (`package.json` local).

services/scrapper

Scraping de tribunais (`tribunal-scraper.ts`), depende de Playwright. Uso operacional separado da API principal.

services/document-generator

Geração de peças (`peca-generator.service.ts`); tipos em `types.ts`.

Estes pacotes evoluem de forma independente da API Nest; integração em produção pode exigir chamadas HTTP internas, filas ou empacotamento como biblioteca — avaliar caso a caso.